

Yes — I'll start with **Week 1** using the slides and practical files you uploaded:

- **Session 1:** FBA and sklearn revisited
- **Session 2:** Model data requirements and learning curves
- **Practical 1:** Orange widgets to Python, Tasks 1–3 and solutions

Below are your **exam-focused Week 1 notes** in the two formats you requested.

Week 1 — Introduction and Recap

Plus Real-World Model Use and Data Challenges

Part A — Structured & Formal Summary

1. Purpose of Week 1

Week 1 is a bridge from **Foundational Business Analytics** into **Machine Learning and Advanced Analytics**. The main purpose is not to introduce many new algorithms, but to make sure you can move from simple graphical workflows, such as Orange, into code-based workflows using Python, pandas and sklearn.

The module stresses that machine learning is not just “choosing a model.” You must understand:

- what kind of problem you are solving,
- what data you have,
- what assumptions your model makes,
- how the model learns,
- how the model can overfit,
- how to evaluate it fairly,
- and how to decide whether the model is suitable for the real-world task.

The slides also emphasise that this is a difficult module and cannot be passed by last-minute memorisation. The exam is closed book, with only pre-submitted handwritten notes allowed, and students are expected to understand concepts deeply rather than rely on ChatGPT or copied code.

2. Prediction Does Not Only Mean “Future”

A key conceptual point from the first lecture is:

Prediction means making an informed guess about something unknown.

This does not necessarily mean predicting the future. For example:

- predicting whether a company is already financially risky,
- predicting whether a banknote is authentic,
- predicting wine quality,
- predicting customer churn,
- predicting whether a transaction is fraudulent.

In all cases, the model is estimating an unknown target value or class from known input features. The unknown may be in the future, but it can also be something already true but not directly observed.

3. Basic Machine Learning Workflow

A standard supervised machine learning workflow is:

1. **Define the problem type**

Decide whether the task is classification, regression, clustering, association, prediction, etc.

2. **Define the features**

Features are the measurable input variables used to describe each observation.

3. **Choose a model**

Select an algorithm based on the problem, data type, expected relationship, available data size, and interpretability needs.

4. **Train the model**

Use labelled data to estimate model parameters.

5. **Evaluate the model**

Test performance on unseen data or through resampling methods such as cross-validation.

6. **Use the model for new cases**

Apply the trained model to new observations to generate predictions.

The lecture connects this workflow to the older Orange-style flow diagrams, then shows that the same logic can be implemented in Python.

4. Features, Instances and Targets

In a standard supervised learning dataset:

- **Rows** represent instances, samples or observations.
- **Columns** represent features, attributes or dimensions.
- One column is usually the **target, class, label** or **output feature**.
- The remaining columns are the **input features**.

Example: in the Iris dataset shown in the lecture, each row is a flower, input features include measurements such as sepal and petal dimensions, and the target is the species/class.

In Python, this is commonly represented as:

- `X` = input feature table
- `y` = output feature / target column

This distinction is essential because sklearn models expect the input and output to be passed separately.

5. Why Move from Orange to Python?

Orange is useful for learning the structure of machine learning workflows because it allows visual connections between widgets. However, the lecture makes clear that Orange is not enough for advanced analytics.

The main limitations are:

5.1 Orange cannot handle large datasets well

The slides explicitly show that Orange may fail or become impractical when the dataset is too large to download or process in the graphical interface.

5.2 Orange has a limited model ecosystem

Orange provides a set of available widgets, but Python provides access to a much wider range of tools, including more advanced models, deep learning libraries, time-series methods and specialised analytics packages.

5.3 Orange hides important details

Orange passes data between widgets in the background. In Python, you must explicitly pass data objects between functions. This requires more understanding, but it also gives you more control and more opportunity to inspect the data at every stage.

5.4 Real-world ML requires custom workflows

Real-world projects often need:

- custom data cleaning,
- SQL extraction,
- feature engineering,
- unusual evaluation strategies,
- large-scale processing,
- deployment,
- model monitoring,
- documentation reading,
- debugging.

These are easier and more flexible in Python than in a graphical workflow tool.

6. Orange Widgets versus Python Functions

The practical explains that libraries provide **custom functions** and **custom data objects**.

Connecting functions in Python is conceptually similar to connecting widgets in Orange, but the data is passed explicitly rather than invisibly through links.

A simplified mapping is:

Orange Concept	Python / sklearn Equivalent
File widget	<code>pandas.read_csv()</code>
Select Columns widget	selecting/dropping columns in pandas
Learner widget	creating an sklearn model object
Test & Score widget	<code>cross_val_score()</code> or <code>cross_validate()</code>
Predictions widget	model <code>.predict()</code> method
Output / Save Data	writing files using pandas or Python

The lecture states that Orange widget use and Python function/object use follow similar categories: data loading, preprocessing/manipulation, models, evaluation, visualisation and output.

7. Data Objects in Python

In standard machine learning tasks, data is usually stored in a table-like structure.

Two common Python representations are:

7.1 NumPy array

A NumPy array is a fast, simple two-dimensional numerical structure. It is efficient but contains less metadata.

7.2 pandas DataFrame

A pandas DataFrame wraps a two-dimensional table but also includes:

- column names,
- data types,
- indexes,
- summary methods,
- filtering methods,
- transformation methods.

The practical states that the module will mainly use pandas DataFrames because they are convenient for summarising and manipulating data for machine learning tasks.

8. Reading Data Correctly

A major Week 1 skill is learning to load data correctly and check that it has been interpreted properly.

When replacing the Orange File widget, Python uses functions such as:

```
pandas.read_csv()
```

Important parameters include:

- `header=None` when the file has no header row,
- `names=[...]` to provide column names during loading,
- `sep=','` or `delimiter=','` to define separators,
- `dtype=...` to manually control data types,
- `skiprows=[...]` to skip metadata rows.

The practical stresses that simply loading a file is not enough. You should preview the data, check headings, check inferred types, and compare against the data documentation.

9. Example: Bankruptcy Prediction Practical

The demo task is to predict whether a company will go bankrupt within two years using a Polish companies bankruptcy dataset.

The workflow shows how to:

1. Load the dataset using `pandas.read_csv()`.
2. Preview the data using `.head()`.
3. Check whether column names and types are correct.
4. Add meaningful column names.
5. Separate the target variable from input features.
6. Create sklearn model objects.
7. Evaluate models using cross-validation.

The output feature is named `status`, with:

- `1` = bankrupt / target class,
- `0` = operating / non-target class.

The practical notes that sklearn assumes `1` is the target class and `0` is the non-target class in binary classification, which is why the output can be left unchanged in this case.

10. Selecting Input Features and Output Feature

In Python, the output feature can be selected directly:

```
output_feature = data.status
```

The input features can be created by dropping the output column:

```
input_features = data.drop('status', axis=1)
```

The practical explains that `axis=1` means dropping a column, while `axis=0` would mean dropping a row. It also notes that `.drop()` returns a copy and does not modify the original DataFrame unless explicitly assigned or done in place.

This is the Python equivalent of Orange's Select Columns widget.

11. Creating Models in sklearn

In sklearn, model creation and model training are separate steps.

For example:

```
rf = RandomForestClassifier()
```

```
lg = LogisticRegression()
```

At this stage, these are **untrained model objects**. They contain the chosen algorithm and fixed hyperparameters, but they have not learned from data yet.

The practical explains that this is similar to configuring a learner widget in Orange. The model parameters can be changed, but early in the module default parameters are often left as they are before deeper hyperparameter tuning is introduced later.

12. Hyperparameters versus Learned Parameters

The lecture distinguishes between:

Hyperparameters

These are fixed before training. They control the structure or behaviour of the algorithm.

Examples:

- number of trees in a random forest,
- maximum tree depth,
- regularisation strength,
- kernel choice in SVM,
- number of neighbours in k-nearest neighbours.

Learned parameters

These are estimated from data during training.

Examples:

- regression coefficients in logistic regression,
- split rules in a decision tree,

- tree structures in a random forest,
- weights in a neural network.

The lecture explains this using the idea that a model has a fixed structure that can be manually altered through hyperparameters, while the algorithm learns some components from data.

13. Evaluation and Cross-Validation

The Orange Test & Score widget is replaced in Python by sklearn functions such as:

```
cross_val_score()
```

The practical explains that Test & Score coordinates repeated splitting of the data into training and testing sets, evaluates a model under a chosen resampling strategy, and averages the results. In Python, `cross_val_score()` performs this role.

Important points:

- `cross_val_score()` evaluates one model at a time.
- It returns one score per fold.
- The parameter `cv=10` means 10-fold cross-validation.
- The parameter `scoring='accuracy'`, `scoring='precision'`, or `scoring='recall'` chooses the evaluation measure.
- If comparing models, use the same splits for each model where possible.

The practical uses:

```
KFold(n_splits=10, shuffle=True, random_state=0)
```

and for classification in Task 3:

```
StratifiedKFold(n_splits=10, shuffle=True, random_state=0)
```

Stratified folds are especially useful in classification because they preserve class proportions across folds.

14. Training Data versus Unseen Test Data

A major warning in Week 1 is that evaluating a model on the same data used for training is bad practice.

If a model is tested on the training data:

- it may look artificially accurate,
- it may reward memorising noise,
- it may not represent real-world performance.

If a model is tested on unseen data:

- memorised noise no longer helps,

- the score is closer to expected real-world performance,
- overfitting is more visible.

The lecture states that overfitted training-set properties are correct for the training data but incorrect for unseen test data.

15. Underfitting and Overfitting

15.1 Underfitting

Underfitting occurs when the model is too simple to capture the underlying relationship in the data.

Signs of underfitting:

- poor training performance,
- poor test performance,
- overly simple boundary or prediction function,
- high bias.

Example: using a straight line when the real pattern is curved.

15.2 Overfitting

Overfitting occurs when the model learns noise or accidental properties of the training set rather than the underlying general relationship.

The lecture defines overfitting as modelling noise or properties of the training set rather than the underlying general relationship.

Signs of overfitting:

- very strong training performance,
- much weaker test performance,
- complex model boundary,
- high variance,
- poor generalisation.

15.3 Balanced fit

A good model is complex enough to capture the true relationship but not so complex that it memorises random noise.

16. Evaluation Measures

Week 1 revises the confusion matrix and evaluation measures.

For binary classification:

Reality / Prediction	Predicted Positive	Predicted Negative
Actually Positive	True Positive	False Negative
Actually Negative	False Positive	True Negative

The lecture's fire-alarm example illustrates that different mistakes have different costs. A false alarm may be annoying, but failing to detect a real fire may be catastrophic.

16.1 Accuracy

Plain formula:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{FN} + \text{TN})$$

Use when correct prediction of every outcome is roughly equally important and classes are reasonably balanced.

16.2 Balanced Accuracy

Plain formula:

$$\text{Balanced accuracy} = \text{mean per-class sensitivity}$$

Use when both classes matter and one class is significantly more common than the other.

16.3 Precision

Plain formula:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Precision asks: **Of the cases predicted as positive, how many were actually positive?**

Use when false positives are costly.

16.4 Sensitivity / Recall

Plain formula:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Recall asks: **Of the actual positives, how many did the model find?**

Use when false negatives are costly.

16.5 Specificity

Plain formula:

$$\text{Specificity} = \text{TN} / (\text{FP} + \text{TN})$$

Specificity asks: **Of the actual negatives, how many did the model correctly identify?**

Use when failing to predict the non-target class is costly.

17. Regression Evaluation Measures from Practical 1

Task 2 uses a wine quality regression problem with:

- Random Forest Regressor,
- Dummy Regressor using the mean,
- 10-fold cross-validation,
- MAE and MSE.

The solution uses `cross_val_score()` with:

- `scoring='neg_mean_absolute_error'` ,
- `scoring='neg_mean_squared_error'` .

sklearn returns negative values for some loss metrics because its scoring framework assumes "higher is better," so the results are multiplied by `-1` or printed as negative means to recover normal error values. In the solution, Random Forest performs better than the mean baseline on both cross-validation and new unseen test data.

18. Classification Evaluation from Practical 1

Task 3 evaluates whether banknotes are authentic using:

- Random Forest Classifier,
- Logistic Regression,
- Dummy / Majority Classifier,
- 10-fold StratifiedKfold,
- accuracy,
- precision,
- recall.

The task explicitly asks students to evaluate Random Forest, Logistic Regression and Constant/Majority Learner, and to sketch the sklearn workflow in advance.

The solution shows:

- Logistic Regression accuracy around 98.98%,
- Random Forest accuracy around 99.20%,
- Majority Learner accuracy around 55.54%,
- Majority Learner precision/recall problems because it may predict no positive samples.

This is important because it shows that a high accuracy baseline can still be useless for the target class. It also demonstrates why precision and recall matter alongside accuracy.

19. Model Comparison: "Which Classifier Is Best?"

The second lecture begins with the question:

Which classifier is the best?

The answer is not a single model. The lecture uses the “no free lunch” idea: no one model is best for all problems. The correct answer is not just “it depends,” but **“it depends on what?”**

Model comparison should consider:

- decision boundary shape,
- data type support,
- missing value handling,
- scaling requirements,
- robustness to correlated features,
- robustness to irrelevant features,
- ability to output probabilities,
- interpretability,
- binary versus multiclass classification,
- training speed,
- prediction speed,
- assumptions,
- hyperparameters,
- learned parameters,
- overfitting control,
- data requirements,
- learning curve behaviour.

20. Model Capabilities versus Model Use Requirements

The lecture separates two types of model knowledge.

20.1 Capabilities

These are things the model can or cannot do.

Examples:

- Can it model nonlinear decision boundaries?
- Can it handle nominal features?
- Can it handle missing values?
- Can it produce class probabilities?
- Can it do multiclass classification?

20.2 Requirements for correct use

These are things the analyst must know to use the model properly.

Examples:

- What assumptions does it make?
- What parameters does it learn?
- What hyperparameters must be set?
- How does it overfit?
- How much data does it need?
- Does it need scaling?
- Does it suffer from irrelevant or correlated features?

This distinction matters because a model may technically run on the data but still be a poor conceptual choice.

21. Logistic Regression Example

The lecture uses logistic regression as an example of model comparison criteria.

Key points:

- Structure: linear decision boundary.
- Assumption: classes are linearly separable given the input features.
- Learned parameters: regression parameters.
- Overfitting control: regularisation through penalisation of regression parameters.
- Important hyperparameter: strength of the penalty.

The lecture notes that these properties can be known by memorising them, but better understanding comes from thinking through how the model reacts in different situations.

22. Data Requirements Depend on the Algorithm and the Problem

The amount of data required depends on two main things:

22.1 Complexity of the problem

If the true relationship is simple, less data may be enough.

If the data population is hard to separate, the true boundary is a complex function, and more parameters are needed. Learning more parameters requires more data.

22.2 Complexity of the learning algorithm

More flexible algorithms can model more complex relationships, but they also often require more data to avoid overfitting.

A simple model may require less data but may underfit a complex problem.

A complex model may fit the training set well but may overfit if the dataset is small.

23. Learning Curves

A learning curve shows how model performance changes as the training set size increases.

The lecture explains that with very small training sets, models may overfit heavily, even simple classifiers. As more data is added, overfitting reduces and the training and test curves should move closer together.

23.1 Training curve

The training score usually starts high because the model can fit a small dataset easily.

As training data increases, training performance may decrease or stabilise because the model has to fit more varied data.

23.2 Test / validation curve

The test or validation score usually starts lower.

As training data increases, the model generalises better and the test score improves.

23.3 Convergence

The two curves should converge towards a stable level.

If both curves converge at poor performance, the model may be too simple or the features may be insufficient.

If there is a large gap between training and test performance, the model may still be overfitting.

24. Why Learning Curves Matter for Model Selection

Learning curves help decide what to do next.

Case 1: High training score, low validation score

Likely problem: overfitting.

Possible responses:

- collect more data,
- simplify the model,
- increase regularisation,
- reduce irrelevant features,
- use cross-validation,
- try a less flexible model.

Case 2: Low training score, low validation score

Likely problem: underfitting.

Possible responses:

- use a more flexible model,
- improve features,
- add nonlinear transformations,
- reduce regularisation,
- check whether the target is learnable from available features.

Case 3: Curves have converged and performance is still poor

Likely problem: model or data limitation.

Possible responses:

- better features,
- new data sources,
- different algorithm,
- reconsider the problem formulation.

Case 4: Validation score still improving with more data

Likely response:

- more data may help.

25. Practical Exam Lessons from Week 1

For the exam, Week 1 is likely to appear as conceptual explanation rather than pure coding.

You should be able to explain:

- why Python is needed instead of Orange,
- how an Orange flow maps to Python/sklearn,
- why documentation matters,
- how to separate input features and output target,
- why evaluating on training data is misleading,
- what overfitting and underfitting mean,
- how to choose evaluation measures,
- why a majority baseline is useful,
- why no single model is always best,
- how model choice depends on data and problem complexity,
- how learning curves guide next steps.

Part B — Q&A Summary

1. What is the main purpose of Week 1?

Week 1 revises core FBA machine learning concepts and moves them into Python/sklearn. It focuses on understanding workflows, evaluation, overfitting, documentation, and model comparison rather than simply memorising algorithms.

2. What does "prediction" mean in this module?

Prediction means making an informed guess about something unknown. It does not only mean predicting the future. A model can predict an unknown class, label, value or outcome even if the event has already happened.

3. What is the basic supervised machine learning structure?

A supervised dataset contains input features and an output target. The model learns the relationship between the input features and the target from labelled examples, then uses that relationship to predict the target for unseen examples.

4. What is a feature?

A feature is an input variable that describes an observation. In a table, features are usually columns.

5. What is an instance?

An instance is one observation, sample or data point. In a table, instances are usually rows.

6. What is a target or output feature?

The target is the variable the model is trying to predict. In classification it is the class label. In regression it is a numerical value.

7. Why is Orange useful?

Orange is useful because it visually shows the structure of a machine learning workflow. It helps beginners understand the flow from data loading to feature selection, model training, evaluation and prediction.

8. Why is Orange not enough for advanced machine learning?

Orange is limited with large datasets, has a restricted set of available models, hides many details of data handling, and is less flexible for custom real-world workflows. Python gives more control, scalability and access to wider libraries.

9. What is the Python equivalent of connecting Orange widgets?

In Python, you connect functions and objects manually by passing data explicitly between them. For example, you load data with pandas, select columns with DataFrame operations, create sklearn model objects, and evaluate them with sklearn functions.

10. What is the practical advantage of Python passing data explicitly?

It forces you to understand the data format, but it also lets you inspect, summarise, debug and transform the data at any point in the workflow.

11. What data structure is mainly used in this module?

The module mainly uses pandas DataFrames because they store table-like data with column names, data types and useful methods for inspection and manipulation.

12. What is a NumPy array?

A NumPy array is a fast numerical data structure, often used internally by machine learning libraries. It is efficient but does not carry as much metadata as a pandas DataFrame.

13. What does `pandas.read_csv()` replace from Orange?

It replaces the File widget. It loads CSV data into Python.

14. Why must you check the data after using `read_csv()` ?

Because the file may have no header, unusual separators, skipped metadata rows or incorrect inferred data types. You must check the preview, headings and data documentation.

15. What does `header=None` mean?

It tells pandas that the file does not contain a header row, so pandas should not treat the first row as column names.

16. What does `skiprows` do?

It tells pandas to skip specified rows while reading the file. This is useful when files include metadata rows or Orange-specific formatting.

17. What does `data.head()` do?

It shows the first few rows of the DataFrame so you can check whether the data loaded correctly.

18. What is the Python equivalent of Orange's Select Columns widget?

Selecting the target column and dropping it from the input feature set. For example, use one column as `y` and use `.drop(target_column, axis=1)` to create `X`.

19. What does `axis=1` mean in `.drop()` ?

It means drop a column. `axis=0` would mean drop a row.

20. Why do we separate `X` and `y` ?

sklearn models generally expect the input features and target labels to be passed separately. `X` contains the predictors and `y` contains the outcome to learn.

21. What is a model object in sklearn?

A model object is an untrained representation of an algorithm with chosen hyperparameters. For example, `RandomForestClassifier()` creates a random forest model object before it has learned anything from data.

22. What is the difference between creating a model and fitting a model?

Creating a model sets up the algorithm and hyperparameters. Fitting a model trains it on data and estimates the learned parameters.

23. What are hyperparameters?

Hyperparameters are settings chosen before training. They control model structure or learning behaviour, such as number of trees, tree depth or regularisation strength.

24. What are learned parameters?

Learned parameters are estimated from the training data. Examples include regression coefficients, decision tree splits and neural network weights.

25. What is overfitting?

Overfitting occurs when a model learns noise or accidental properties of the training set instead of the underlying general relationship.

26. What is underfitting?

Underfitting occurs when a model is too simple to capture the real pattern in the data.

27. Why is testing on training data bad?

Because the model may have memorised training data patterns or noise. This gives artificially high performance and does not show how well the model will generalise to new data.

28. What is unseen test data?

Unseen test data is data that was not used to train the model. It gives a more realistic estimate of real-world model performance.

29. What does `cross_val_score()` do?

It repeatedly trains and tests a model using a resampling strategy such as cross-validation and returns one score per fold.

30. Why use cross-validation?

Cross-validation gives a more stable estimate of model performance than a single train-test split because the model is evaluated across multiple splits.

31. What is 10-fold cross-validation?

The data is split into 10 parts. The model is trained on 9 parts and tested on the remaining part. This is repeated 10 times so each part is used once as the test fold.

32. Why use the same folds when comparing models?

Using the same folds makes the comparison fairer because each model is tested on the same data splits.

33. What is StratifiedKFold?

StratifiedKFold is a cross-validation method that preserves class proportions in each fold. It is useful for classification, especially when classes are imbalanced.

34. What is a DummyClassifier or Majority Learner?

It is a baseline model that makes simple predictions, often based on the most frequent class. It is useful for checking whether a real model is actually better than a trivial strategy.

35. Why can a majority classifier have misleading accuracy?

If one class is much more common, predicting the majority class can achieve high accuracy while completely failing to identify the minority or target class.

36. What is accuracy?

Accuracy is the proportion of all predictions that are correct.

Plain formula:

$$\text{Accuracy} = (TP + TN) / (TP + FP + FN + TN)$$

37. When is accuracy useful?

Accuracy is useful when classes are balanced and all types of mistakes have similar cost.

38. What is precision?

Precision measures how many predicted positives are actually positive.

Plain formula:

$$\text{Precision} = TP / (TP + FP)$$

39. When is precision important?

Precision is important when false positives are costly.

Example: wrongly accusing a genuine transaction of being fraud.

40. What is recall?

Recall measures how many actual positives are found by the model.

Plain formula:

$$\text{Recall} = TP / (TP + FN)$$

41. When is recall important?

Recall is important when false negatives are costly.

Example: failing to detect disease, fraud, fire or customer churn.

42. What is specificity?

Specificity measures how many actual negatives are correctly identified.

Plain formula:

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

43. What is balanced accuracy?

Balanced accuracy averages performance across classes. It is useful when one class is much more common than the other.

44. What is MAE?

MAE stands for Mean Absolute Error. It measures the average absolute difference between predicted and actual values.

45. What is MSE?

MSE stands for Mean Squared Error. It squares errors before averaging, so larger errors are penalised more heavily.

46. Why does sklearn sometimes use negative error values?

sklearn's scoring system assumes higher scores are better. Since errors are losses and lower is better, sklearn returns negative error values for scoring functions such as negative MAE or negative MSE.

47. What does "no free lunch" mean in model selection?

It means no single model is best for every problem. A model that works well on one dataset may perform badly on another.

48. Why is "it depends" not a good enough answer?

Because it is vague. A good analyst must explain what it depends on: data size, feature types, decision boundary shape, noise, class balance, interpretability, assumptions and evaluation goals.

49. What should you compare when choosing models?

You should compare capabilities, assumptions, data requirements, robustness, interpretability, overfitting behaviour, speed, output type and learning curve behaviour.

50. What is a linear decision boundary?

A linear decision boundary separates classes using a straight line, flat plane or hyperplane in the feature space.

51. What kind of decision boundary does logistic regression learn?

Logistic regression learns a linear decision boundary.

52. What assumption does logistic regression make in the lecture?

The lecture states that logistic regression assumes classes are linearly separable given the input features.

53. How does logistic regression deal with overfitting?

It can use regularisation, which penalises large regression parameters.

54. What are correlated features?

Correlated features are input variables that contain overlapping or related information.

55. Why do irrelevant features matter?

Irrelevant features can add noise, increase model complexity, reduce interpretability and make some algorithms overfit or perform worse.

56. Why does problem complexity affect data requirements?

A complex true relationship requires more parameters to model. Learning more parameters reliably requires more data.

57. Why does algorithm complexity affect data requirements?

More flexible algorithms can represent more complex patterns, but they usually need more data to avoid overfitting.

58. What is a learning curve?

A learning curve shows how model performance changes as the amount of training data increases.

59. What usually happens to training performance as data size increases?

Training performance often decreases or stabilises because the model can no longer simply fit a tiny dataset.

60. What usually happens to validation/test performance as data size increases?

Validation or test performance usually improves because the model learns a more general pattern.

61. What does a large gap between training and validation performance suggest?

It suggests overfitting. The model performs well on training data but poorly on unseen data.

62. What does low training and low validation performance suggest?

It suggests underfitting, poor features, an unsuitable model, or a problem that is difficult to learn from the available data.

63. What does it mean if both curves converge at poor performance?

It suggests a model limitation or data limitation. More of the same data may not help much unless features or model choice improve.

64. What should you do if the validation curve is still improving as more data is added?

Collecting more data may help because the model has not yet reached its performance plateau.

65. What is the main exam message of Week 1?

Do not think of machine learning as "try models and pick the highest score." You must understand the workflow, data, model assumptions, evaluation measures, baselines, overfitting, and learning curves to justify your decisions.

Week 1 High-Yield Exam Checklist

Before moving to Week 2, make sure you can confidently explain:

- Orange workflow to Python/sklearn workflow.
- Difference between input features and output target.
- How `read_csv`, `.head()`, `.drop()`, model objects and `cross_val_score()` fit into the workflow.
- Why training-set evaluation is misleading.

- Difference between underfitting and overfitting.
- Accuracy, balanced accuracy, precision, recall and specificity.
- Why majority baselines matter.
- Why no single classifier is always best.
- What model comparison should depend on.
- What learning curves show.
- How to decide whether to collect more data, change model, change features or reduce overfitting.